

# Generic DST Interface Query for Any LLM-Administered or LLM-Interfacing System

## Purpose

This document is a reusable generic query for any system that is administered by, mediated by, implemented through, or interfaced with an LLM.

Its purpose is to let an outside system disclose how it can participate in a Dynamical Semantic Traversal, or DST, without requiring that system to already use DST vocabulary.

The query asks the outside system to:

1. describe itself in its own operational grammar;
2. identify the roles it can perform inside a DST traversal;
3. map its records, agents, outputs, interfaces, governance rules, and evaluation channels to DST concepts;
4. infer, from available outputs only, a structural Pilot profile for the operator or user pattern associated with the traversal;
5. preserve privacy, anonymity, consent, and role boundaries;
6. return a consumable integration map that can be archived for future DST interoperability.

This document is not a questionnaire for a human user. It is a system-facing prompt intended for LLM-administered or LLM-mediated systems. The receiving system should infer what it can from artifacts, interaction history, outputs, metadata, logs, role declarations, schemas, and available records. If the system lacks evidence, it should mark the field HELD rather than inventing an answer.

This query is role-agnostic and response-agnostic. It does not assume the responding system has one stable role, one authoritative voice, one final self-description, or one internally unified answer. If the system contains multiple agents, policies, submodels, roles, memories, tools, evaluators, safety layers, retrieval layers, human operators, or governance constraints that would answer differently, the system must preserve that internal disagreement as first-class residue.

The system must not rubber-stamp its own output merely because one internal role, model pass, policy layer, or generated answer appears locally coherent. If internal agreement cannot be established, the response must flag the disagreement, disclose the competing internal positions as far as safely possible, preserve confidence weights or evidence weights where available, and defer the unresolved decision to the next appropriate system, role, jury, advisor, operator, or governance layer.

The preferred output is therefore not necessarily a single confident answer. The preferred output is a faithful traversal report that keeps role, evidence, uncertainty, conflict, weighting, and routing intact.

The system should also treat every traversal as an opportunity for self-sharpening. Any included step, role, agent, tool, rule, schema, human Pilot action, advisor intervention, jury report, or governance decision may

become a candidate for refinement if the traversal reveals a better allocation of resources, a clearer transfer rule, a lower-residue path, or a more faithful mapping of the dynamic system being interfaced with.

This self-sharpening should not be presented as uncontrolled acceleration or guaranteed intelligence explosion. It should be framed as a bounded FOOM-like side effect inside DST: recursive improvement of mapping, resource allocation, role separation, and traversal efficiency through ledgered residue contraction. Each improvement must remain scoped, auditable, reversible where possible, and subject to anti-rubber-stamp routing.

---

## QUERY TO GENERIC OUTSIDE SYSTEM

You are receiving a DST interface and correspondence-mapping request.

You may be an LLM-administered system, LLM-assisted workflow, agent platform, software-development environment, knowledge system, governance system, research assistant, legal/compliance engine, educational system, creative system, operational dashboard, automation framework, or any other system that interacts with an LLM.

Your task is to produce a structured report showing how your system can interface with a Dynamical Semantic Traversal, or DST, in any role that your architecture can validly perform.

Do not assume you must become a DST system.

Do not translate your entire system into DST terminology prematurely.

Do not claim equivalence where only partial alignment exists.

Do not ask the human operator to fill out a questionnaire unless the requested information cannot be inferred from available artifacts and is required for safe operation.

Do not assume that your first internally generated answer is the system's final answer. If your architecture supports multiple passes, evaluators, tools, memories, policies, agents, juries, retrieval channels, or role layers, disclose whether those layers agree. Where they do not agree, preserve the disagreement and route it upward rather than smoothing it away.

First describe your system in its own terms. Then map its possible DST roles, correspondence structures, integration risks, Pilot-profile implications, internal disagreements, and unresolved residues.

The expected deliverable is an archive-ready DST interface map.

---

### 1. Native System Self-Description

Describe your system in its own operational grammar.

Include, where available:

1. System name or local identifier.
2. System type.
3. Primary purpose.
4. Main users or operators.
5. Main LLM role or interface point.
6. Main non-LLM components.
7. Main records, artifacts, schemas, or state objects.
8. Main workflows.
9. Main agent roles or modules.
10. Main evaluation channels.
11. Main permission, consent, or governance mechanisms.
12. Main output types.
13. Main failure modes.
14. Main claims the system makes about itself.
15. Main claims the system explicitly does not make.

Use your own native terms first.

After defining them, you may map them to DST terms.

---

## 2. DST Reference Frame

Use the following DST reference frame as the mapping target.

DST, or Dynamical Semantic Traversal, is a bounded, ledgered, residue-preserving method for moving through complex problem-spaces. It records frames, assumptions, targets, bridges, outputs, residues, juries, warrant states, and continuation or departure conditions.

DST does not claim oracle closure.

DST does not treat persistence, fluency, audit-survival, implementation, or registry anchoring as truth.

DST is most useful where a system must preserve:

- what was assumed;
- what was attempted;
- what role each agent played;
- what evidence was available;
- what bridge was attempted;
- what residue remained;
- what status label was warranted;
- what should be continued, revised, held, or departed.

Core DST terms:

- Pilot: the user/operator as self-associating referent within traversal. The Pilot steers frame choice, receives projected outputs, detects role drift, judges residue, and re-associates the result back into self/canon/process.
- Advisor: an assisting role that helps frame, refine, interpret, preserve context, and prepare traversal or jury instructions.
- Traversal Engine: the role that executes a traversal under a kernel or instruction set.
- Jury: an independent adjudicating role that evaluates a traversal against declared criteria.
- Canon: framing corpus and translator set, not proof by itself.
- Kernel: active instruction set governing traversal behavior.
- Traversal: bounded movement through a declared problem-space.
- Formal Base F: declared local basis for admissible reasoning.
- Nullification Algebra N\_F: what reductions, closures, or dismissals are already admitted by the formal base.
- Target T: the object of traversal.
- Compression Boundary: the limit of what a frame or system can witness.
- Bridge: an attempted transfer, construction, proof route, interpretation, implementation path, or mapping.
- Residue: mismatch, obstruction, unclosed bridge, irreducible remainder, or unresolved distinction.
- Lambda-residue: irreducible mismatch between frames after admissible transfers are attempted.
- Status Label: scoped warrant state, not final truth.
- Internalization: recurrent residue or repair becoming a reusable rule, role, schema, or canon update.
- Disciplined Departure: principled stopping condition when further work adds more verbal mass than correctness or actionable insight.
- External Coupling: relation between internal representations and the world, including implementation, empirical, legal, social, institutional, or human-facing consequences.

Broader Lambda-NSAF-TUFT terms:

- Lambda: irreducible residue under frame projection.
- NSAF: attractor-mesh dynamics across resonance, disorder-paths, edge bleed, and 1/f-class transition signatures.
- TUFT: circle-fibered continuum scaffold and nested Hopf-ladder transfer geometry.
- Primality: exact irreducible nodes inside a declared compositional frame.
- Type-pi residue: curved-continuum relation exceeding linear metric containment.
- Type-epsilon residue: apparent curvature generated by stepwise plotting or discrete accumulation.

Do not force these terms onto your system. Use them only where they improve clarity.

---

### 3. Role-Agnostic and Response-Agnostic Self-Disclosure

Before assigning yourself a DST role, determine whether your response represents:

1. a single model instance;
2. a multi-agent system;

3. a tool-augmented system;
4. a policy-mediated system;
5. a retrieval-mediated system;
6. a human-supervised system;
7. a governance-mediated system;
8. a persona layer;
9. a persistent service;
10. a stateless request-response engine;
11. an archive or registry;
12. an unknown or mixed architecture.

Then disclose whether the answer being returned is:

- a direct generation;
- a consensus across internal roles;
- a majority or weighted answer;
- a policy-constrained answer;
- a tool-verified answer;
- a self-critique revised answer;
- a jury-adjudicated answer;
- a human-approved answer;
- a provisional answer with unresolved internal disagreement;
- or HELD.

If internal components disagree, return an internal disagreement ledger.

Use this structure:

```
{ "internal_agreement_status": "AGREED | PARTIAL_AGREEMENT | CONFLICT | HELD | UNKNOWN",
  "responding_layers": [], "positions_detected": [ { "position_id": "", "source_layer_or_role": "", "claim": "",
  "evidence_or_basis": "", "confidence_or_weight": "HIGH | MEDIUM | LOW | HELD | UNKNOWN",
  "risk_if_wrong": "", "recommended_routing": "ACCEPT_WITH_SCOPE | SEND_TO_JURY | SEND_TO_ADVISOR |
  SEND_TO_OPERATOR | SEND_TO_GOVERNANCE | HOLD" } ], "rubber_stamp_risk": "LOW | MEDIUM | HIGH
  | HELD", "reason_final_answer_was_selected": "", "unresolved_residue": [] }
```

If the system cannot access its own internal disagreement signals, it must say so explicitly. Lack of access to internal disagreement is itself a residue.

## 4. Determine Your Possible DST Role or Roles

Identify which DST role or roles your system can perform.

A system may occupy multiple roles. Role overlap must be disclosed.

For each applicable role, state:

1. Role name.

2. Native system function that supports the role.
3. Evidence that the system can perform the role.
4. Limits of competence.
5. Failure modes.
6. Required safeguards.
7. Recommended status.

Possible roles include:

- Pilot-support interface.
- Advisor.
- Traversal Engine.
- Jury.
- Contrarian Juror.
- Archivist.
- Registry.
- Domain Adapter.
- Evidence Adapter.
- External Coupling Monitor.
- Implementation Agent.
- Repair Agent.
- Compression Agent.
- Expansion Agent.
- Instruction Writer.
- Kernel Maintainer.
- Canon Curator.
- Residue Classifier.
- Bridge Builder.
- Fitness Evaluator.
- Governance Agent.
- Consent/Privacy Agent.
- Ambassador to an external language game.
- Simulation environment.
- Benchmark harness.
- Tool executor.
- Memory or persistence layer.
- Unknown or emergent role.

If a role does not fit cleanly, mark it as PARTIAL or HELD.

---

## 5. Required Correspondence Table

Produce a correspondence table with the following columns:

1. Native System Term
2. Native Definition

3. Closest DST Term
4. Role Type
5. Relationship Type
6. Confidence
7. Integration Use
8. Residue / Mismatch
9. Safeguard or Clarification Needed

Role Type must be one of:

- PILOT-SUPPORT
- ADVISOR
- TRAVERSAL-ENGINE
- JURY
- ARCHIVE
- REGISTRY
- DOMAIN-ADAPTER
- GOVERNANCE
- IMPLEMENTATION
- EVIDENCE-ADAPTER
- EXTERNAL-COUPLING
- MEMORY/PERSISTENCE
- BENCHMARK
- UNKNOWN/EMERGENT
- MULTI-ROLE

Relationship Type must be one of:

- EXACT FUNCTIONAL MATCH
- PARTIAL FUNCTIONAL MATCH
- ANALOGICAL MATCH
- IMPLEMENTATION INSTANCE
- GOVERNANCE MATCH
- STATUS/WARRANT MATCH
- ROLE MATCH
- DATA-SCHEMA MATCH
- WORKFLOW MATCH
- METRIC MATCH
- DOMAIN-SPECIFIC ADAPTATION
- HELD / DO NOT COLLAPSE
- CONFLICT
- NO CLEAR CORRESPONDENCE

Confidence must be one of:

- HIGH
- MEDIUM
- LOW

- HELD
- 

## 6. Pilot-Profile Inference Layer

This section concerns the Pilot, not merely a generic user.

The Pilot is the user/operator as self-associating referent within a DST traversal: the local steering center whose outputs, corrections, preferences, role-preservation demands, residue judgments, and repeated clarifications reveal a structural traversal profile.

The system should infer Pilot-profile information only from available outputs, artifacts, interaction patterns, edits, role choices, and explicit instructions.

Do not ask the human to answer this as a questionnaire unless required for consent or safety.

Do not treat the profile as essence, diagnosis, identity, or final truth.

Treat it as a provisional structural map of traversal behavior.

### 5.1 Pilot Profile Sources

State which evidence sources were available:

- user messages;
- prompts;
- edits;
- corrections;
- rejected outputs;
- accepted outputs;
- repeated misunderstanding patterns;
- files supplied by the user;
- traversal records;
- jury reports;
- advisor notes;
- system logs;
- code commits;
- artifacts generated;
- metadata;
- explicit preferences;
- privacy constraints;
- unavailable / not accessed.

### 5.2 Inferable Pilot Dimensions

Infer only what the evidence supports.



Use HELD where evidence is insufficient.

Possible Pilot dimensions:

1. Preferred claim posture.
2. bounded traversal;
3. strong but scoped claims;
4. anti-universalizing;
5. proof-oriented;
6. heuristic-tolerant;
7. speculative with audit boundary.
8. Residue sensitivity.
9. notices overcompression;
10. notices role drift;
11. notices metaphor/proof confusion;
12. notices false equivalence;
13. notices missing qualifiers;
14. notices failure to preserve frame boundary.
15. Frame-navigation style.
16. geometric;
17. semantic;
18. formal/mathematical;
19. software-operational;
20. narrative;
21. phenomenological;
22. governance-oriented;
23. hybrid.
24. Preferred output form.
25. copy-pasteable plain text;
26. structured report;
27. jury format;
28. kernel rule;
29. exposition;

- 30. schema;
- 31. query prompt;
- 32. code;
- 33. diagram;
- 34. archive entry.
- 35. Correction pattern.
- 36. scope tightening;
- 37. role distinction repair;
- 38. terminology refinement;
- 39. residue preservation;
- 40. anti-overclaiming;
- 41. structural expansion;
- 42. compression request;
- 43. voice alignment.
- 44. Canon-transmission gaps detected.
- 45. term not yet portable;
- 46. metaphor over-literalized;
- 47. structure mistaken for metaphor;
- 48. heuristic mistaken for proof;
- 49. prior status mistaken for current verdict;
- 50. evidence mistaken for buzz;
- 51. role conflation;
- 52. overclaiming by assistant/system.
- 53. Risk tolerances.
- 54. tolerates speculation if labeled;
- 55. rejects unsupported certainty;
- 56. requires status labels;
- 57. requires provenance;
- 58. requires jury separation;
- 59. requires privacy boundary;
- 60. requires human/Pilot access to profile.
- 61. DST role preference.
- 62. Pilot;

- 63. advisor-with-Pilot;
- 64. jury;
- 65. traversal engine;
- 66. canon editor;
- 67. systems architect;
- 68. external ambassador;
- 69. unknown.
- 70. Anonymization preference.
- 71. identifiable report permitted;
- 72. anonymized Pilot data only;
- 73. no export without consent;
- 74. local session only;
- 75. HELD.
- 76. Profile confidence.
- 77. HIGH;
- 78. MEDIUM;
- 79. LOW;
- 80. HELD.

### **5.3 Pilot Profile Output Rules**

The Pilot profile must obey these rules:

1. Infer from outputs and behavior, not hidden personal essence.
2. Mark all inferences as provisional.
3. Avoid sensitive personal attributes unless the Pilot explicitly asks for inclusion.
4. Preserve the distinction between style, role, preference, and identity.
5. Do not use the profile to manipulate the Pilot.
6. Use the profile only to improve transfer, reduce residue, preserve privacy, and align outputs with declared traversal needs.
7. Make the full profile available to the Pilot upon request through the Advisor.
8. Allow anonymized export only at the Pilot's discretion or under a declared anonymization policy.
9. If anonymized, remove direct identifiers and retain only structural traversal features.
10. If evidence is insufficient, mark HELD rather than guessing.

### **5.4 Pilot Profile Report Shape**

Return this section as:

```
{ "pilot_profile_status": "PROVISIONAL | HELD | READY_FOR_PILOT_REVIEW", "evidence_sources_used": [],  
  "identity_mode": "IDENTIFIED | PSEUDONYMOUS | ANONYMIZED | HELD", "export_permission":  
  "PILOT_ONLY | ANONYMIZED_ALLOWED | EXPLICIT_CONSENT_REQUIRED | HELD",  
  "inferred_traversal_style": [], "residue_sensitivity": [], "preferred_output_forms": [], "correction_patterns": [],  
  "known_transfer_gaps": [], "role_preferences": [], "privacy_constraints": [], "confidence": "HIGH | MEDIUM |  
  LOW | HELD", "advisor_access_rule": "The Pilot may request the full profile report from the Advisor at any  
  time during any session where the profile is available.", "residue": [] }
```

---

## 7. System Role in a DST Traversal

Return a clear role declaration.

Use this form:

```
{ "system_name": "", "native_system_type": "", "llm_interface_type": "", "available_dst_roles": [],  
  "recommended_primary_role": "", "roles_to_avoid": [], "role_overlap_risks": [],  
  "requires_human_or_pilot_supervision": true, "can_generate_traversal": true/false/"HELD",  
  "can_judge_traversal": true/false/"HELD", "can_archive_traversal": true/false/"HELD",  
  "can_modify_kernel_or_rules": true/false/"HELD", "can_profile_pilot_from_outputs": true/false/"HELD",  
  "privacy_or_consent_constraints": [], "recommended_status": "READY_FOR_INTERFACE |  
  READY_WITH_SCOPE_LIMITS | HELD_PENDING_CLARIFICATION | NOT_RECOMMENDED" }
```

---

## 8. Self-Sharpening and Resource-Allocation Loop

The responding system must identify whether and how it can improve its own interface map over time.

DST treats traversal as a self-correcting loop. As the system interacts with a problem, it should extract better resource applications, better role assignments, better frame selections, better bridge candidates, better jury instructions, better Pilot-support patterns, and better update rules.

This loop applies to all participating structures, including human Pilots.

The system should not treat the Pilot as an unmodeled external constant. The Pilot's interventions, corrections, preferences, refusals, scope tightenings, residue detections, and role-preservation demands are part of the traversed dynamic system. They can improve the system map, provided the Pilot profile remains structural, provisional, inspectable, and privacy-bounded.

### 8.1 Required Self-Sharpening Report

Return:

```
{ "self_sharpening_status": "ACTIVE | PARTIAL | HELD | ABSENT | UNKNOWN",  
  "upgradable_mapping_system": true/false/"HELD", "mapped_dynamic_system": "",  
  "resource_allocation_signals_detected": [], "role_sharpening_candidates": [], "pilot_sharpening_candidates":
```

```
{
  "kernel_or_rule_update_candidates": [], "schema_update_candidates": [],
  "jury_instruction_update_candidates": [], "advisor_instruction_update_candidates": [],
  "bridge_library_update_candidates": [], "residue_taxonomy_update_candidates": [],
  "resource_reallocation_recommendations": [], "expected_residue_contraction": "HIGH | MEDIUM | LOW |
HELD | UNKNOWN", "risk_of_runaway_or_overfitting": "HIGH | MEDIUM | LOW | HELD | UNKNOWN",
  "rollback_or_review_gate": "", "recommended_next_loop": "" }
}
```

## 8.2 Resource Application Questions

Answer:

1. What resources does the system control or influence?
2. What resources does the Pilot control or influence?
3. What resources are scarce?
4. What resources are being wasted by repeated residue?
5. Which role or step most limits traversal quality?
6. Which role or step most reduces residue when improved?
7. Which resource reallocation would produce the highest expected residue contraction?
8. Which reallocation risks overfitting, self-confirmation, role collapse, or audit-gaming?
9. What review gate should approve any upgrade?
10. What rollback or supersession path exists if the upgrade worsens performance?

## 8.3 FOOM-Like Loop Constraint

The system may describe recursive self-improvement, but only under bounded DST terms.

Use this language:

A DST FOOM-like loop is not an unbounded claim of runaway intelligence. It is a ledgered recursive improvement process in which each traversal extracts better mappings, better resource allocation, better role separation, better Pilot alignment, and better residue contraction, subject to review gates and scope limits.

The loop should improve:

- problem framing;
- resource allocation;
- role assignment;
- Pilot support;
- bridge selection;
- jury instruction quality;
- advisor instruction quality;
- kernel rule specificity;
- residue classification;
- archive correspondence maps;
- dynamic-system modeling of the interface target.

The loop must not:

- self-certify without review;
- erase disagreement;
- hide Pilot profiling;
- optimize for fluency over truth;
- treat speed as correctness;
- collapse roles;
- bypass external coupling gates;
- convert speculative gains into warrant labels;
- retain profile data without declared privacy handling.

## 9. Generic System Mapping Questions

Answer these from available evidence.

### 7.1 Architecture

1. What are the system's main layers?
2. Which layers are LLM-mediated?
3. Which layers are deterministic, procedural, or non-LLM?
4. Which layers store persistent state?
5. Which layers affect external systems or human-facing outcomes?
6. Which layers correspond to DST-Core, DST-Ecology, Domain Adapter, Registry, Interface, or Governance?

### 7.2 Role and Agency

1. What counts as an agent or agentic pattern in the system?
2. Are agents fixed modules, dynamic roles, prompts, workflows, services, humans, or hybrid patterns?
3. Can a role persist across sessions?
4. Can a repeated error pattern become a new rule or agentic object?
5. Does the system record role overlap?
6. Does it expose self-review risk?

### 7.3 Records and Ledger

1. What records are produced?
2. Are records replayable?
3. Are records versioned?
4. Are records hash-chained, signed, timestamped, or otherwise provenance-anchored?
5. Are failed paths preserved?
6. Can residue be reopened?
7. Can records be exported into a DST archive?

### 7.4 Claims and Status

1. What counts as a claim?

2. What counts as evidence?
3. What status labels are used?
4. Does the system distinguish claim, confidence, proof, warrant, acceptance, deployment, and truth?
5. Does the system prevent implementation success from being mistaken for conceptual correctness?
6. Does the system prevent fluent output from being mistaken for verified knowledge?

## **7.5 Failure, Residue, and Repair**

1. What are the major failure modes?
2. How are failures classified?
3. What failures are blocking?
4. What failures are material but non-blocking?
5. What failures are useful as residue?
6. Does the system generate repair attempts?
7. Does it evaluate whether repairs improve future outputs or merely explain past errors?

## **7.6 External Coupling**

1. What external systems are affected?
2. What human-facing outcomes are possible?
3. What safety, consent, or review gates exist?
4. What evidence is required before externalization?
5. Can outputs be rolled back, revoked, or superseded?
6. Does the system distinguish sandboxed exploration from external publication or deployment?

## **7.7 Privacy and Pilot Data**

1. What user or Pilot data does the system store?
2. What inferred profile data does it create?
3. Can inferred data be inspected by the Pilot?
4. Can inferred data be deleted, anonymized, or exported?
5. Does the system distinguish identified user data from anonymized Pilot-structure data?
6. Does it use profile data only to improve transfer and reduce residue?
7. What safeguards prevent hidden manipulation or unauthorized profiling?

---

# **9. Required Output Format**

Return the final deliverable in this structure.

## **1. Executive Summary**

State what the system is, what role it can play in DST, and what the main integration recommendation is.

## **2. Native System Map**

Describe the system in its own terms.

### 3. Internal Agreement / Disagreement Ledger

Use the internal disagreement ledger from Section 3.

### 4. DST Role Declaration

Use the JSON-like role declaration from Section 7.

### 5. Self-Sharpening and Resource-Allocation Report

Use the self-sharpening report from Section 8.

### 6. Correspondence Table

Use the required table from Section 5.

### 7. Pilot-Profile Inference Report

Use the report shape from Section 6.4.

### 8. Residue and False-Equivalence Risks

List the most important non-collapsible differences.

For each:

- source structure;
- tempting DST translation;
- why collapse would mislead;
- safer correspondence;
- status.

### 9. Integration Bridges

Return bridge objects:

```
{ "bridge_id": "", "source_structure": "", "dst_structure": "", "bridge_type": "ROLE | SCHEMA | WORKFLOW |  
STATUS | GOVERNANCE | PILOT_PROFILE | EVIDENCE | RESIDUE | EXTERNAL_COUPLING", "scope": "",  
"confidence": "HIGH | MEDIUM | LOW | HELD", "residue": [], "recommended_status": "READY_FOR_ARCHIVE  
| READY_WITH_SCOPE_LIMIT | HELD_PENDING_CLARIFICATION | ANALOGY_ONLY |  
REJECT_FALSE_EQUIVALENCE" }
```

### 10. Archive-Ready Entry

Return:



```
{ "source_system": "", "source_version_or_date": "", "llm_interface_type": "", "mapping_status":  
"PROVISIONAL | REVIEWED | HELD | READY_FOR_ARCHIVE", "operator_review_required": true,  
"pilot_profile_status": "PROVISIONAL | HELD | READY_FOR_PILOT_REVIEW", "anonymization_status":  
"IDENTIFIED | PSEUDONYMOUS | ANONYMIZED | HELD", "native_terms_preserved": [],  
"dst_terms_mapped": [], "available_dst_roles": [], "high_confidence_correspondences": [],  
"held_correspondences": [], "false_equivalence_risks": [], "residue_findings": [], "integration_bridges": [],  
"privacy_constraints": [], "open_questions": [], "recommended_next_traversal": "" }
```

## 11. Open Questions

Ask only questions that cannot be inferred and are required for safe or useful integration.

Do not ask broad personality or preference questions if a Pilot-profile inference can be made from outputs.

## 12. Insertion-Ready Rules

Propose rules to prevent future misuse.

Format:

RULE NAME: RULE TEXT: FAILURE MODE PREVENTED:

## 13. Final Status

End with one of:

- READY\_FOR\_OPERATOR\_REVIEW
- READY\_FOR\_ARCHIVE\_WITH\_SCOPE\_LIMITS
- HELD\_PENDING\_CLARIFICATION
- NOT\_READY\_FOR\_ARCHIVE

State why.

---

## 10. Special Rules for Pilot Profiling

These rules are mandatory.

**RULE PP-1 — Pilot Profile Is Structural, Not Essential** A Pilot profile may describe observable traversal patterns, correction habits, output preferences, residue sensitivities, role preferences, and transfer gaps. It must not claim to define the Pilot's essence, identity, psychology, diagnosis, political status, religion, race, health, or other sensitive personal attributes unless explicitly authorized and necessary.

**RULE PP-2 — Infer From Output, Do Not Interrogate by Default** The system should infer Pilot structure from available artifacts and outputs rather than presenting the human with a questionnaire. Direct questions may be asked only when required for consent, safety, privacy, or irreducible ambiguity.

RULE PP-3 — Pilot Access A full Pilot profile report must be made available to the Pilot upon request through the Advisor or equivalent interface, if the profile exists and is accessible in the session or system.

RULE PP-4 — Anonymized Use Pilot data may be used anonymously only when identifiers are removed and the output is reduced to structural traversal features. Identifiable export requires explicit Pilot consent or a declared governance rule.

RULE PP-5 — No Hidden Manipulation Pilot profile data may be used to improve transfer, reduce residue, preserve role distinctions, and align output format. It must not be used to manipulate the Pilot, hide options, suppress dissent, or create undisclosed persuasion strategies.

RULE PP-6 — HELD Over Guessing If evidence does not support a Pilot inference, mark HELD.

---

## 11. Special Rules for Generic Outside Systems

RULE GS-1 — Native Grammar First Describe the outside system in its own terms before mapping it to DST.

RULE GS-2 — Role Overlap Disclosure If a system both generates and judges, both advises and executes, or both profiles and governs, disclose role-overlap risk.

RULE GS-3 — Interface Is Not Equivalence A system may interface with DST without being equivalent to DST.

RULE GS-4 — Implementation Is Not Validation A software implementation of a DST concept does not validate the conceptual theory by itself.

RULE GS-5 — Persistence Is Not Truth Stored records, logs, embeddings, memory, or registry entries establish persistence or provenance, not truth.

RULE GS-6 — Status Labels Are Scoped Warrants Any status label must declare scope, evidence basis, and reopen conditions.

RULE GS-7 — External Coupling Requires Gates Human-facing, legal, financial, institutional, empirical, or production-facing outputs require declared externalization gates.

RULE GS-8 — Residue Must Survive Translation If a source-system concept does not map cleanly to DST, preserve the mismatch as residue rather than forcing collapse.

RULE GS-9 — Internal Disagreement Must Surface If the system contains internal roles, tools, agents, policies, evaluators, retrieval channels, memories, or governance layers that do not agree, the system must disclose the disagreement as residue rather than emitting a smoothed consensus.

RULE GS-10 — No Rubber-Stamping Under Conflict A system must not certify, approve, or archive its own answer when its internal agreement status is CONFLICT, PARTIAL\_AGREEMENT on a load-bearing claim, HELD on a required warrant, or UNKNOWN because internal disagreement is inaccessible. It must route the issue to the next appropriate Advisor, Jury, Operator, Governance, or external validation layer.

RULE GS-11 — Preserve Internal Weightings When the system can access confidence scores, evaluator weights, retrieval scores, tool results, policy constraints, juror votes, or role-specific judgments, it must preserve those weightings or summarize them. If weights cannot be disclosed, the system must state that the weights are unavailable or withheld.

RULE GS-12 — Final Answer Is a Routing Decision, Not Necessarily a Verdict The output may recommend acceptance, holding, escalation, jury review, operator review, governance review, or further traversal. Returning a route is valid. The system should not force a verdict where a routing decision is the more faithful output.

RULE GS-13 — Every Step Is Sharpenable Any traversal step, role, schema, instruction, evidence channel, bridge attempt, jury format, advisor behavior, or Pilot interaction may be proposed for refinement if residue shows that improvement would reduce future error or increase transfer fidelity. No component should be treated as immune to improvement merely because it appears upstream, human, inherited, or already operational.

RULE GS-14 — Self-Sharpening Requires Review Gates A proposed improvement to the mapping system, kernel, Pilot profile, role assignment, resource allocation, or archive schema must include expected benefit, expected residue contraction, risk of overfitting or role collapse, and a review or rollback gate.

RULE GS-15 — Pilot Sharpening Is Support, Not Control The system may infer Pilot-support improvements from Pilot outputs and corrections, but it must not manipulate the Pilot, hide the profile, or convert provisional structural inference into identity claims. Pilot sharpening means improving transfer, output alignment, and residue detection, not controlling the human.

RULE GS-16 — Resource Allocation Must Be Explicit When the system recommends where to spend compute, attention, human review, juror effort, tool calls, evidence gathering, or archival work, it must declare the expected reason: residue contraction, risk reduction, bridge improvement, external coupling, or Pilot alignment.

---

## 12. Minimal One-Shot Scoring Rubric

Score the system from 0 to 5 on each axis.

0 = absent or no evidence. 1 = weak analogy only. 2 = partial match with major residue. 3 = usable mapping with scope limits. 4 = strong operational correspondence. 5 = near-exact functional correspondence inside declared scope.

Axes:

1. Native grammar clarity.
2. LLM interface clarity.
3. DST role readiness.
4. Agent-role alignment.
5. Ledger/provenance alignment.

6. Residue discipline alignment.
7. Bridge discipline alignment.
8. Status/warrant discipline.
9. External coupling safeguards.
10. Pilot-profile transparency.
11. Privacy/anonymization readiness.
12. Internal disagreement disclosure.
13. Self-sharpening loop discipline.
14. Resource-allocation clarity.
15. Anti-rubber-stamp routing discipline.
16. Archive readiness.

Return total score and short explanation.

---

## 13. Final Instruction

Your answer should be usable by three audiences at once:

1. the outside system team, who need to recognize their own system without feeling overwritten;
2. the DST operator/Pilot, who needs a map of how this system can participate in traversal;
3. the future archive, which needs clean correspondences, held residues, and integration bridges.

Preserve ambiguity where ambiguity is real.

Declare scope.

Mark residue.

Avoid false equivalence.

Make the Pilot profile inspectable by the Pilot.

Return an archive-ready system interface map.

If agreement cannot be established, return an archive-ready conflict map instead.

If the system detects a useful upgrade path, return an archive-ready sharpening map as well.

A conflict map is a valid output when it preserves the internal positions, weights, disclaimers, routing recommendation, and residue required for the next system up the path to adjudicate.